組合語言 上機大抄

正面：模板＋作業 1–3＋小考

模板 通用模板（每行寫 C／STDCALL，.model 寫哪個都能跑）

```
.386
.model flat, c
option casemap:none
includelib kernel32.lib
includelib msvcrt.lib
includelib legacy_stdio_definitions.lib
includelib ucrt.lib
ExitProcess PROTO STDCALL :DWORD
SetConsoleOutputCP PROTO STDCALL :DWORD
printf PROTO C :VARARG
scanf  PROTO C :VARARG
time   PROTO C :DWORD
srand  PROTO C :DWORD
rand   PROTO C
system PROTO C :VARARG
.data
n         dword ?
formatIn  byte "%d", 0
formatOut byte "%d", 0Ah, 0 ; 0Ah 換行
msgPause  byte "pause", 0
```

```
.code
main PROC C
  INVOKE SetConsoleOutputCP, 65001 ; 中文
  INVOKE scanf, OFFSET formatIn, OFFSET n ; 位址
  INVOKE printf, OFFSET formatOut, n ; 值
  INVOKE system, ADDR msgPause ; 停住
  INVOKE ExitProcess, 0
main ENDP
; 自己的 PROC 寫這裡
END main
```

上機 VS 流程與錯誤

空白專案（英文路徑）→ 專案右鍵「建置相依性 → 建置自訂」勾 **masm** → 平台 **x86** → 加入 main.asm → main.asm 右鍵「屬性 → 一般 → **項目類型**」改成 **Microsoft Macro Assembler** → Ctrl+Shift+B → **Ctrl+F5**

A1000 中文路徑 ・ **LNK2019** 少 lib ・ **A2006** 拼錯／少 PROTO ・ **A2008/A2151** 撞保留字（name size type nop outs）・ **A2070** mem,mem／大小不合 ・ x64 改 x86

工具 push/call、程序範本、常用片段

```
push n                    ; = INVOKE printf,f,n
push OFFSET f             ; 由右至左 push
call printf
add esp, 8                ; C：個數×4
F PROC STDCALL x:DWORD, y:DWORD
  mov eax, x              ; 回傳值放 eax
  ret                     ; 自動 ret 8
F ENDP                    ; push y/push x/call F
G PROC C x:DWORD          ; 呼叫後 add esp,4
; 有號 jg jge jl jle｜無號/字元 ja jae jb jbe
; 除法 xor edx,edx / div ecx → 商 eax 餘 edx
; 亂數 a~b：rand / div (b-a+1) / add edx, a
; 奇偶 test eax,1 / jz｜大小寫 or 20h / and 0DFh
```

作業1 Q1–Q5 高階語言轉組語

```
i    dword 123
j    dword ?
a    dword 1
b    dword 1
res  dword ?
x    dword 2
y    dword 1
r2   dword ?
vals dword 2, 4, 6, 8
resx dword ?
```

```
; Q1  j = i; i++; j--;
 mov eax, i
 mov j, eax               ; mem,mem 不行，經過 eax
 inc i
 dec j
; Q2  res = ++a - b; b--;
 inc a
 mov eax, a
 sub eax, b
 mov res, eax
 dec b
; Q3  r2 = (y*4 + x + 1) * 2（不用乘法）
 mov eax, y
 shl eax, 2               ; ×4
 add eax, x
 inc eax
 shl eax, 1               ; ×2
 mov r2, eax
; Q5  resx = vals[0]+…+vals[3]
 mov eax, vals[0]
 add eax, vals[4]         ; DWORD 一格 4
 add eax, vals[8]
 add eax, vals[12]
 mov resx, eax
```

作業2 Q1 a1[i] += i，再 loop 加總

```
a1 dword 10,20,30,40,50,60,70,80,90,100
sumResult dword ?
```

```
  mov ecx, LENGTHOF a1
  mov esi, 0              ; esi = 索引
L1: mov eax, a1[esi*4]
  add eax, esi
  mov a1[esi*4], eax      ; a1[i] += i
  inc esi
  loop L1
  mov ecx, LENGTHOF a1
  mov esi, 0
  mov eax, 0
L2: add eax, a1[esi*4]
  inc esi
  loop L2
  mov sumResult, eax
```

作業2 Q2 16 位元最大／最小值與 OF

```
OF_1 dword ?
OF_2 dword ?
```

```
  mov ax, 7FFFh           ; 最大正數 +32767
  add ax, 1               ; 溢位
  pushfd                  ; 旗標推進堆疊
  pop eax
  and eax, 0800h          ; 只留 OF (bit 11)
  mov OF_1, eax           ; 800h = OF 是 1
  mov bx, 8000h           ; 最小負數 -32768
  sub bx, 1
  pushfd
  pop eax
  and eax, 0800h
  mov OF_2, eax
```

作業2 Q4 兩陣列相加，再向左旋轉

```
arr1 dword 1, 2, 3, 4
arr2 dword 10, 20, 30, 40
arr3 dword 4 DUP(?)
ARRAY_SIZE = LENGTHOF arr1
```

```
  mov ecx, ARRAY_SIZE
  mov esi, 0              ; esi = 位元組位移
add_loop:
  mov eax, arr1[esi]
  add eax, arr2[esi]
  mov arr3[esi], eax
  add esi, 4
  loop add_loop
  mov edx, arr3[0]        ; 備份第一個
  mov ecx, ARRAY_SIZE - 1
  mov esi, 0
rotate_loop:
  mov eax, arr3[esi + 4] ; 拿右邊鄰居
  mov arr3[esi], eax
  add esi, 4
  loop rotate_loop
  mov arr3[esi], edx      ; 第一個放最後
```

作業2 Q5 費氏數列前 11 項

```
fib dword 11 DUP(?)
```

```
  mov fib[0], 1
  mov fib[4], 1
  mov ecx, 9              ; 還要算 9 項
  mov esi, 8              ; 從 fib[2] 開始
fib_loop:
  mov eax, fib[esi - 4]
  add eax, fib[esi - 8]
  mov fib[esi], eax       ; f[i]=f[i-1]+f[i-2]
  add esi, 4
  loop fib_loop
```

小考 ArraySum 程序（11/2）＋ STDCALL 程序（10/19）

```
arrS DWORD 10000h,20000h,3000h,400000h,50000h
fmt5 BYTE "%d is a multiple of 5",10,0
```

```
  mov esi, OFFSET arrS    ; 用暫存器傳參數
  mov ecx, LENGTHOF arrS
  call ArraySum           ; 結果在 eax
  push 25
  call p_div5             ; stdcall：不用 add esp
```

程序：寫在 main ENDP 之後

```
ArraySum PROC
  push esi
  push ecx
  mov eax, 0
AS1: add eax, [esi]       ; esi 是位址，不 ×4
  add esi, TYPE DWORD
  loop AS1
  pop ecx                 ; 反順序 pop
  pop esi
  ret
ArraySum ENDP
p_div5 PROC STDCALL nn:DWORD
  mov eax, nn             ; = [ebp+8]
  xor edx, edx
  mov ecx, 5
  div ecx
  cmp edx, 0              ; 餘數
  jne over1
  INVOKE printf, OFFSET fmt5, nn
over1: ret                ; 自動 ret 4
p_div5 ENDP
```

作業3 Q1 n+…+5+3+1

```
n dword ?
sumResult3 dword ?
formatIn byte "%d", 0
formatOut byte "%d", 0Ah, 0
```

```
  INVOKE scanf, OFFSET formatIn, OFFSET n
  mov eax, 0
  mov ecx, n
odd_sum_loop:
  add eax, ecx
  sub ecx, 2
  cmp ecx, 0
  jg odd_sum_loop
  mov sumResult3, eax
  INVOKE printf, OFFSET formatOut, sumResult3
```

作業3 Q2 1*1+…+n*n（巢狀、不用乘法）

```
sumSq dword 0
```

```
  INVOKE scanf, OFFSET formatIn, OFFSET n
  mov ecx, n
  mov ebx, 1
outer_loop:
  push ecx                ; 存外圈 ecx
  mov ecx, ebx            ; 內圈跑 ebx 次
  mov edx, 0
inner_loop:
  add edx, ebx            ; ebx 加 ebx 次
  loop inner_loop
  add sumSq, edx
  inc ebx
  pop ecx                 ; 還原外圈
  loop outer_loop
  INVOKE printf, OFFSET formatOut, sumSq
```

作業3 Q3 質數判斷

```
msgPrime byte "Prime", 0Ah, 0
msgNotPrime byte "Not Prime", 0Ah, 0
```

```
  INVOKE scanf, OFFSET formatIn, OFFSET n
  cmp n, 1
  jle not_prime_label
  cmp n, 2
  je is_prime_label
  mov ebx, 2
check_loop:
  mov eax, n
  mov edx, 0              ; 除法前清 edx
  div ebx
  cmp edx, 0              ; 餘數 0 → 整除
  je not_prime_label
  inc ebx
  cmp ebx, n
  jl check_loop
is_prime_label:
  INVOKE printf, OFFSET msgPrime
  jmp end_program
not_prime_label:
  INVOKE printf, OFFSET msgNotPrime
end_program:
```

作業3 Q4 大小寫互換 abCD→ABcd

```
strBuf byte 100 DUP(0)
formatS byte "%s", 0
formatOutS byte "%s", 0Ah, 0
```

```
  INVOKE scanf, OFFSET formatS, OFFSET strBuf
  mov esi, OFFSET strBuf
convert_loop:
  mov al, [esi]
  cmp al, 0
  je print_result         ; 字串結尾
  cmp al, 'A'
  jb next_char            ; 字元用 jb/ja 較正確
  cmp al, 'Z'
  jbe to_lower
  cmp al, 'a'
  jb next_char
  cmp al, 'z'
  jbe to_upper
  jmp next_char
to_lower:
  add al, 20h
  jmp store_char
to_upper:
  sub al, 20h
store_char:
  mov [esi], al
next_char:
  inc esi
  jmp convert_loop
print_result:
  INVOKE printf, OFFSET formatOutS, OFFSET strBuf
```

必看 寫完檢查

call 會弄壞 eax ecx edx ・ loop 用 ecx ・ mem,mem 不行 ・ ADDR 只在 INVOKE ・ esi 是位址→[esi]，是索引→arr[esi*4] ・ 除法前清 edx ・ 每個分支都 jmp ・ push/pop 成對 ・ 最後 ExitProcess

組合語言 上機大抄

背面：作業 4＋小考與範例（全部實測可跑）

作業4-1 猜數字（重玩、0／-1 結束）

```
cnt dword 1
mxR dword ?
tar dword ?
guess dword ?
askLimit BYTE "max (0/-1 quit): ", 0
askGuess BYTE "guess: ", 0
showCnt  BYTE "correct! %d times", 10, 0
gameOver BYTE "game over", 10, 0
showBig  BYTE "too big", 10, 0
showSmol BYTE "too small", 10, 0
```

```
  INVOKE time, 0
  INVOKE srand, eax       ; 洗牌一次
while_loop:
  INVOKE printf, ADDR askLimit
  INVOKE scanf, ADDR formatIn, ADDR mxR
  cmp mxR, 0
  je game_over
  cmp mxR, -1
  je game_over
  call rand
  xor edx, edx
  div mxR                 ; 餘數 0~mxR-1
  inc edx
  mov tar, edx
  mov cnt, 1              ; 每局重設
guess_loop:
  INVOKE printf, ADDR askGuess
  INVOKE scanf, ADDR formatIn, ADDR guess
  mov eax, guess
  cmp eax, 0
  je game_over
  cmp eax, -1
  je game_over
  cmp eax, tar
  je find_same
  jl find_less
  jg find_greater
find_same:
  INVOKE printf, ADDR showCnt, cnt
  jmp while_loop          ; 重玩
find_less:
  INVOKE printf, ADDR showSmol
  inc cnt
  jmp guess_loop
find_greater:
  INVOKE printf, ADDR showBig
  inc cnt
  jmp guess_loop
game_over:
  INVOKE printf, ADDR gameOver
```

作業4-2 核心：出題、四路分支、記錄錯題、等第

```
mnR dword 1
num dword 1
cntQ dword 1
op dword 1
numA dword ?
numB dword ?
ans dword ?
ansInput dword ?
correct dword 0
wrong dword 0
wCnt dword 4 DUP(0)
wA dword 50 DUP(0)
score dword 0
fmtAdd BYTE "%d) %d + %d = ", 0
fmtSub BYTE "%d) %d - %d = ", 0
fmtMul BYTE "%d) %d * %d = ", 0
fmtDiv BYTE "%d) %d / %d = ", 0
qFmt dword OFFSET fmtAdd, OFFSET fmtSub, OFFSET
fmtMul, OFFSET fmtDiv
gradeA BYTE "A",0
gradeB BYTE "B",0
gradePass BYTE "PASS",0
gradeFail BYTE "FAIL",0
msgRes BYTE "score %d %s",10,0
```

```
  ; mxR、mnR、num 先用 scanf 讀入
q_loop:
  mov eax, cntQ
  cmp eax, num
  jg q_end
  call rand               ; op = rand % 4
  xor edx, edx
  mov ecx, 4
  div ecx
  mov op, edx
  call rand               ; numA = mnR ~ mxR
  mov ecx, mxR
  sub ecx, mnR
  inc ecx
  xor edx, edx
  div ecx
  add edx, mnR
  mov numA, edx
  mov numB, 3             ; (numB 同上產生)
  cmp op, 0
  je do_add
  cmp op, 1
  je do_sub
  cmp op, 2
  je do_mul
do_div:                   ; 先定商，A = B×商
  mov eax, mxR
  xor edx, edx
  div numB
  mov ebx, eax            ; 商最大 = mxR/B
  call rand
  xor edx, edx
  div ebx
  inc edx
  mov ans, edx
  mov eax, numB
  mul edx
  mov numA, eax
  jmp ask
do_add:
  mov eax, numA
  add eax, numB
  mov ans, eax
  jmp ask                 ; 都跳到匯合點
do_sub:
  mov eax, numA
  cmp eax, numB
  jge sub_ok              ; 不可小減大
  mov ebx, numB
  mov numA, ebx
  mov numB, eax
sub_ok:
  mov eax, numA
  sub eax, numB
  mov ans, eax
  jmp ask
do_mul:
  mov eax, numA
  mul numB
  mov ans, eax
ask:
  mov ebx, op             ; 用 op 挑格式字串
  INVOKE printf, qFmt[ebx*4], cntQ, numA, numB
  INVOKE scanf, ADDR formatIn, ADDR ansInput
  mov eax, ansInput
  cmp eax, ans
  jne q_wrong
  inc correct
  jmp q_next
q_wrong:
  mov ebx, wrong          ; 存到第 wrong 格
  mov eax, numA
  mov wA[ebx*4], eax      ; (wB wOp wAns 同法)
  inc wrong
  mov ebx, op
  inc wCnt[ebx*4]         ; 該類錯 +1
q_next:
  inc cntQ
  jmp q_loop
q_end:
  mov eax, correct        ; 分數 = 對×100÷題數
  mov ecx, 100
  mul ecx
  xor edx, edx
  div num
  mov score, eax
  mov ebx, OFFSET gradeFail ; 由差往好升級
  cmp score, 60
  jl show
  mov ebx, OFFSET gradePass
  cmp score, 80
  jl show
  mov ebx, OFFSET gradeB
  cmp score, 90
  jl show
  mov ebx, OFFSET gradeA
show:
  INVOKE printf, ADDR msgRes, score, ebx
```

作業4-2 輸入檢查、各類錯題數、錯題清單

```
userName BYTE 64 DUP(0)
nameFmt  BYTE "%s", 0
rangeFmt BYTE "%d %d", 0
msgAskWrong BYTE "bad range",10,0
wQ   dword 50 DUP(0)
wB   dword 50 DUP(0)
wOp  dword 50 DUP(0)
wAns dword 50 DUP(0)
msgCat BYTE "add %d sub %d mul %d div %d",10,0
msgWrong BYTE " ans %d",10,0
msgName BYTE "%s",10,0
```

```
  INVOKE scanf, ADDR nameFmt, ADDR userName ; 字串
ask_range:
  INVOKE scanf, ADDR rangeFmt, ADDR mxR, ADDR mnR
  mov eax, mnR
  cmp mxR, eax
  jl ask_range_wrong      ; 上限 < 下限
  cmp mnR, 1
  jl ask_range_wrong      ; 下限 < 1（除法會除以 0）
  ; ……出題迴圈……
  INVOKE printf, ADDR msgName, ADDR userName ; %s
給位址
  INVOKE printf, ADDR msgCat, wCnt[0], wCnt[4],
wCnt[8], wCnt[12]
  mov cntQ, 0             ; cnt 用完拿來當索引
list_loop:
  mov eax, cntQ
  cmp eax, wrong
  jge list_end
  mov ebx, cntQ           ; ebx 不會被 printf 改
  mov eax, wOp[ebx*4]
  INVOKE printf, qFmt[eax*4], wQ[ebx*4],
wA[ebx*4], wB[ebx*4]
  INVOKE printf, ADDR msgWrong, wAns[ebx*4]
  inc cntQ
  jmp list_loop
list_end:
  jmp done42
ask_range_wrong:          ; 放在走不到的地方
  INVOKE printf, ADDR msgAskWrong
  jmp ask_range
done42:
```

小考 最大值、push/call 亂數、階乘

```
fmtR BYTE "%d %d",10,0
```

```
; 11/2 ax,bx,cx 最大值 → ax（有號）
  mov ax, 7
  mov bx, 42
  mov cx, 15
  cmp ax, bx
  jg pp1
  xchg ax, bx
pp1: cmp ax, cx
  jg pp2
  xchg ax, cx
pp2: movsx esi, ax        ; esi=42
; 11/2 不用 INVOKE：3~98 亂數 → edx
  push 0
  call time
  add esp, 4
  push eax
  call srand
  add esp, 4
  call rand               ; 結果在 eax
  mov ecx, 96             ; 98-3+1
  mov edx, 0
  div ecx
  add edx, 3
; 範例：階乘（C 慣例）
  push 5
  call Factorial
  add esp, 4              ; C：呼叫者清
  INVOKE printf, OFFSET fmtR, esi, eax
```

程序：寫在 main ENDP 之後

```
Factorial PROC C USES ecx, nF:DWORD
  mov ecx, nF
  mov eax, 1
FL: mul ecx               ; eax = eax*ecx
  loop FL
  ret
Factorial ENDP
```

範例 字串比對 repe cmpsb、堆疊反轉

```
s1 BYTE "Hello",0
s2 BYTE "Hello",0
nm BYTE "Taipei",0
nmLen = ($ - nm) - 1
fmtC BYTE "%d %s",10,0
```

```
  mov esi, OFFSET s1
  mov edi, OFFSET s2
  mov ecx, LENGTHOF s1
  cld                     ; 往後比
  repe cmpsb              ; 一樣就繼續
  mov ebx, 0
  jne cDiff
  mov ebx, 1              ; 1 = 相同
cDiff: mov ecx, nmLen
  mov esi, 0
R1: movzx eax, nm[esi]    ; push 不能推 8 位
  push eax
  inc esi
  loop R1
  mov ecx, nmLen
  mov esi, 0
R2: pop eax               ; 反序拿回
  mov nm[esi], al
  inc esi
  loop R2
  INVOKE printf, OFFSET fmtC, ebx, OFFSET nm
```